Name: **Shrey Pendurkar**
Class: **D15C**
Batch: **B**
Roll No: **38**

MLDL – Experiment 5

Aim

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

---

1. Dataset Source

- Dataset: Heart Disease UCI Dataset
- Kaggle: https://www.kaggle.com/datasets/mragpavank/heart-diseaseuci
- File used: heart.csv

---

2. Dataset Description

The Heart Disease UCI dataset is compiled from several medical centres and aims to predict the presence of heart disease in patients based on clinical and demographic features. It is a standard binary classification benchmark in medical machine learning.

Key properties:

- Dataset size: 303 rows × 14 columns (13 features + 1 target)
- Target variable: target (1 = heart disease present, 0 = absent)
- Class distribution: 165 positive (heart disease), 138 negative (no heart disease) — mildly imbalanced
- Missing values: some entries encoded as '?', imputed with column means

Feature columns:

- age: Age in years
- sex: Sex (1 = male, 0 = female)
- cp: Chest pain type (0–3; typical angina, atypical angina, non-anginal pain, asymptomatic)
- trestbps: Resting blood pressure (mm Hg)
- chol: Serum cholesterol (mg/dl)
- fbs: Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)

- restecg: Resting electrocardiographic results
- thalach: Maximum heart rate achieved
- exang: Exercise-induced angina (1 = yes, 0 = no)
- oldpeak: ST depression induced by exercise relative to rest
- slope: Slope of the peak exercise ST segment
- ca: Number of major vessels (0–3) coloured by fluoroscopy
- thal: Thallium stress test result (1 = normal, 2 = fixed defect, 3 = reversible defect)

---

3. Mathematical Formulation of the Algorithm

Consider a binary classification dataset $\{(x_i, y_i)\}_{i=1}^{n}$ where $x_i \in \mathbb{R}^d$ are feature vectors and $y_i \in \{-1, +1\}$ are class labels. SVM seeks a separating hyperplane $f(x) = w \cdot x + b = 0$ that maximises the margin between classes.

3.1 Hard-Margin SVM (linearly separable data)

For perfectly separable data, SVM solves:

$$\text{Minimise} \quad (1/2)\, ||w||^2$$

$$\text{Subject to} \quad y_i\,(w \cdot x_i + b) \geq 1 \quad \text{for all } i$$

The margin width equals $2 / ||w||$. Maximising the margin is equivalent to minimising $||w||$.

3.2 Soft-Margin SVM (non-separable data)

For real data with overlapping classes, slack variables $\xi_i \geq 0$ allow margin violations at a cost controlled by C:

$$\text{Minimise} \quad (1/2)\, ||w||^2 + C \sum_i \xi_i$$

$$\text{Subject to} \quad y_i\,(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \text{for all } i$$

C controls the bias–variance trade-off: small C allows more violations (wider, smoother margin); large C penalises violations heavily (narrow, tight margin).

3.3 RBF Kernel (Radial Basis Function)

For non-linearly separable data, the kernel trick implicitly maps inputs to a higher-dimensional space. The RBF kernel is:

$$K(x_i, x^x) = \exp(-\gamma\, ||x_i - x^x||^2)$$

where $\gamma > 0$ controls the radius of influence of each training point. Small $\gamma \rightarrow$ broad influence, smooth boundary; large $\gamma \rightarrow$ narrow influence, complex boundary.

3.4 Decision Function

$$f(x) = \text{sign}\left(\sum_i \alpha_i\, y_i\, K(x_i, x) + b \right)$$

where α_i are the Lagrange multipliers obtained from the dual optimisation. Training samples with $\alpha_i > 0$ are the support vectors.

---

4. Algorithm Limitations

1. High Computational Cost on Large Datasets

Training SVM involves solving a quadratic optimisation problem with complexity typically between $O(n^2)$ and $O(n^3)$, where n is the number of training samples. This makes kernel SVMs expensive in time and memory for large datasets, as they must store and operate on an n×n kernel matrix.

2. Limited Interpretability

SVMs, especially with non-linear kernels, are black-box models. The decision boundary is defined in a transformed, high-dimensional feature space, unlike decision trees or logistic regression, which provide human-readable rules or direct coefficient interpretations.

3. Challenges with Overlapping Classes and Noisy Labels

When classes overlap significantly or labels are noisy, many points violate margin constraints. The model relies heavily on slack variables and the choice of C, and the resulting boundary may not generalise well.

4. Feature Scaling Requirement

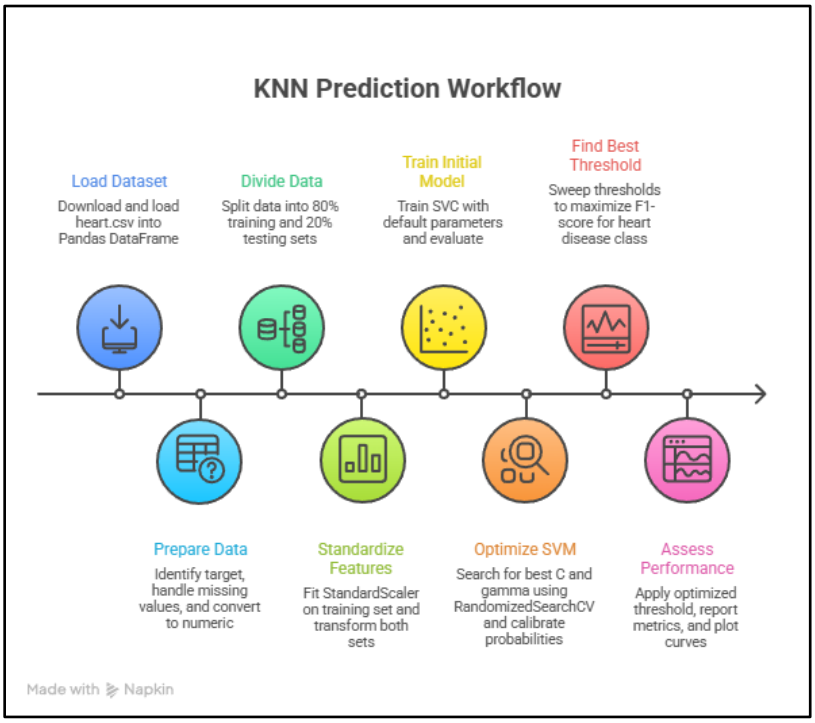
SVM is sensitive to the scale of features because margin width and kernel values depend on distances in feature space. Features with larger numeric ranges dominate the distance calculation. StandardScaler normalization is non-negotiable before training.

---

5. Methodology / Workflow

1. **Data Collection**
   - Load heart.csv from the Kaggle Heart Disease UCI dataset into a Pandas DataFrame.
2. **Target Identification and Preprocessing**
   - Target: target (1 = heart disease, 0 = no heart disease); features: all remaining 13 columns.
   - Replace ‘?’ entries with NaN; coerce columns to numeric; impute missing values with column means.
3. **Train–Test Split**
   - Split into 80% training (242 samples) and 20% testing (61 samples) with stratify=y and random_state=42 to preserve class proportions.
4. **Feature Scaling**
   - Fit StandardScaler on the training set; transform both training and test sets to zero mean and unit variance.

5. **Baseline SVM Model (Without Tuning)**
   - Train SVC(kernel='rbf', class_weight='balanced', random_state=42) with all default hyperparameters (C=1.0, gamma='scale').
   - Evaluate: accuracy, classification report (per-class precision, recall, F1), confusion matrix.
6. **Hyperparameter Tuning (RandomizedSearchCV)**
   - Search over C ~ LogUniform(0.01, 100) and gamma ~ LogUniform(0.0001, 10) using n_iter=20, cv=3, scoring='f1'.
   - Select the best estimator; wrap with CalibratedClassifierCV (sigmoid, cv=3) for calibrated probabilities.
7. **Decision Threshold Optimisation**
   - Sweep thresholds from 0.1 to 0.9 (50 steps); select the threshold that maximises F1-score for the heart disease class.
8. **Final Evaluation**
   - Apply the optimised threshold to probability estimates; report accuracy, classification report, confusion matrix.
   - Plot ROC curve (AUC), Precision–Recall curve (AP), and F1 vs Threshold curve.



---

6. Performance Analysis (Without Hyperparameter Tuning)

Model: SVC(kernel='rbf', class_weight='balanced', random_state=42) — default C=1.0, gamma='scale'

Train/Test Split: 80% / 20% | Test Set Size: 61 samples (28 negative, 33 positive)

Metric	No Heart Disease	Heart Disease	Overall
Precision	0.81	0.80	—
Recall	0.75	0.85	—
F1-Score	0.78	0.82	—
Support	28	33	61
Accuracy	—	—	80.33%

1. **No Heart Disease (Class 0)**

   Precision of 0.81 means 81% of patients predicted healthy are truly healthy. Recall of 0.75 indicates 25% of truly healthy patients are incorrectly flagged as having heart disease — false positives. Confusion matrix: TN=21, FP=7.

2. **Heart Disease (Class 1)**

   Precision of 0.80 and recall of 0.85 show a balanced detection profile. The model correctly identifies 85% of true heart disease cases (28 out of 33). Confusion matrix: TP=28, FN=5. The 5 missed cases represent the highest clinical risk.

3. **Overall**

   The baseline RBF SVM achieves 80.33% accuracy (49/61 correct). The balanced class_weight parameter ensures neither class is systematically ignored despite the mild class imbalance. This is a competitive baseline for a medical dataset of only 303 samples.

---

7. Hyperparameter Tuning

Before Tuning — Default Parameters

Parameter	Default Value
C	1.0
gamma	scale (1 / (n_features × Var(X)))
kernel	rbf
class_weight	balanced
threshold	0.5 (default)

Baseline Test Accuracy: 80.33% | Heart Disease F1: 0.82 | Heart Disease Recall: 0.85

RandomizedSearchCV Configuration:

- C: LogUniform distribution over [0.01, 100]
- gamma: LogUniform distribution over [0.0001, 10]

- n_iter = 20, cv = 3, scoring = ‘f1’ (optimising for heart disease class), random_state = 42
- Post-tuning: CalibratedClassifierCV (method=‘sigmoid’, cv=3) for probability calibration
- Threshold sweep: 50 values from 0.1 → 0.9, selecting argmax F1

Best Parameters Found:

Parameter	Best Value
C	0.147
gamma	0.00679
kernel	rbf
class_weight	balanced
threshold	0.508

Best CV F1 Score: 0.8613 | Best F1 at Optimised Threshold: 0.8333

After Tuning — Results:

Metric	No Heart Disease	Heart Disease	Overall
Precision	0.86	0.77	—
Recall	0.68	0.91	—
F1-Score	0.76	0.83	—
Support	28	33	61
Accuracy	—	—	80.33%

Key Observations:

- Overall test accuracy remains 80.33% — the tuned model does not improve bulk accuracy, but fundamentally reshapes the precision–recall trade-off
- Heart disease recall improved from 0.85 → 0.91 — missed cases (false negatives) dropped from 5 → 3. In a clinical screening context, this is the most important metric: missing a diseased patient is far more dangerous than a false alarm
- The tuned C=0.147 is smaller than the default C=1.0 — more regularisation, a wider margin, fewer support vectors, and a smoother decision boundary better suited to this small dataset
- The tuned gamma=0.00679 is very small — each training point has broad influence, preventing the model from overfitting to individual examples

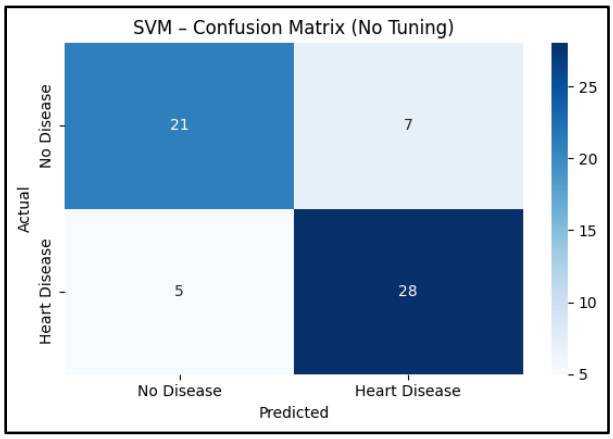
- The optimised threshold (0.508) is nearly identical to the default (0.5), confirming that probability calibration produced well-calibrated estimates and no aggressive threshold shift was needed
- The trade-off: healthy patient recall drops from 0.75 → 0.68 (FP increases from 7 → 9) — acceptable in a screening context where false negatives are costlier than false positives

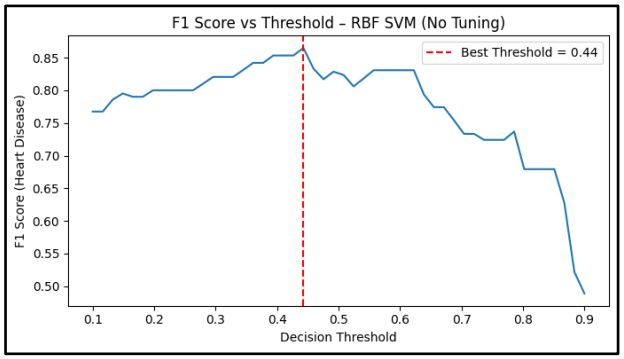
---

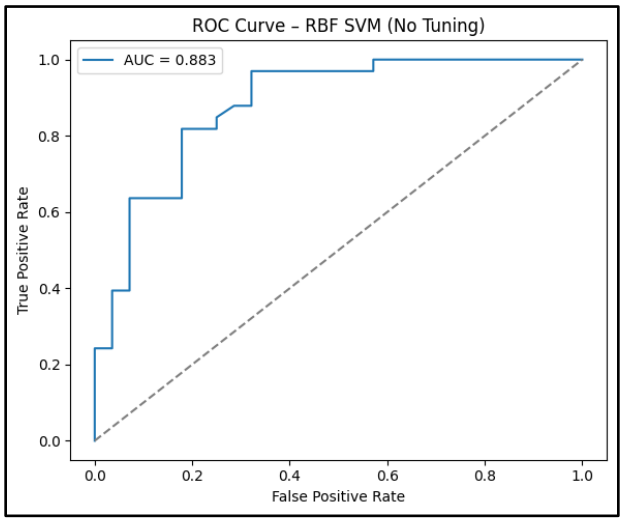
8. Code and Output – Without Tuning

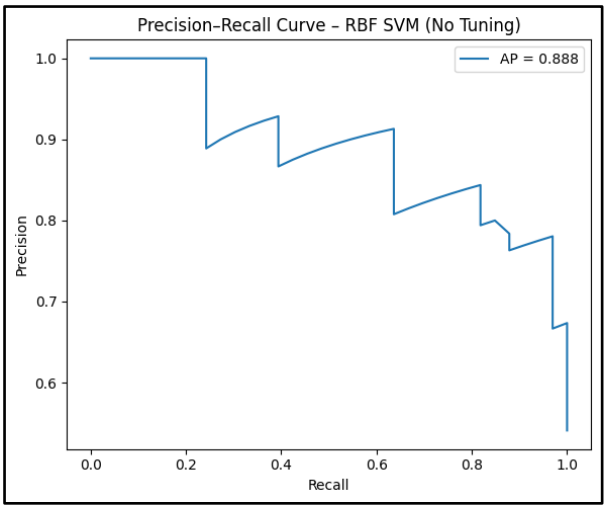
```
# Support Vector Machine – Without Hyperparameter Tuning
# Dataset: Heart Disease UCI (heart.csv from Kaggle)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load dataset
df = pd.read_csv("heart.csv")
print("Initial shape:", df.shape)
print(df.head())

# Clean: replace '?' with NaN, coerce to numeric, fill NaN with column means
df = df.replace("?", np.nan)
df = df.apply(pd.to_numeric, errors="coerce")
df = df.fillna(df.mean(numeric_only=True))

target_col = "target"
X = df.drop(target_col, axis=1)
y = df[target_col]

print("\nClass distribution:\n", y.value_counts())

# Train-test split (80/20, stratified)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
print("\nTrain shape:", X_train.shape, "| Test shape:", X_test.shape)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# Baseline SVM — RBF kernel, class_weight='balanced', all other defaults
svm_base = SVC(kernel='rbf', class_weight='balanced', random_state=42)
svm_base.fit(X_train_scaled, y_train)

# Predictions
y_pred = svm_base.predict(X_test_scaled)

# Performance
acc = accuracy_score(y_test, y_pred)
print("\n========== SVM (No Tuning) ==========")
print(f"Accuracy: {acc * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
```

```
    target_names=["No Heart Disease",
"Heart Disease"]))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
            xticklabels=["No Disease",
"Heart Disease"],
            yticklabels=["No Disease",
"Heart Disease"])
plt.title('SVM – Confusion Matrix (No
Tuning)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()

print("\nDefault Parameters Used:")
print(svm_base.get_params())
```







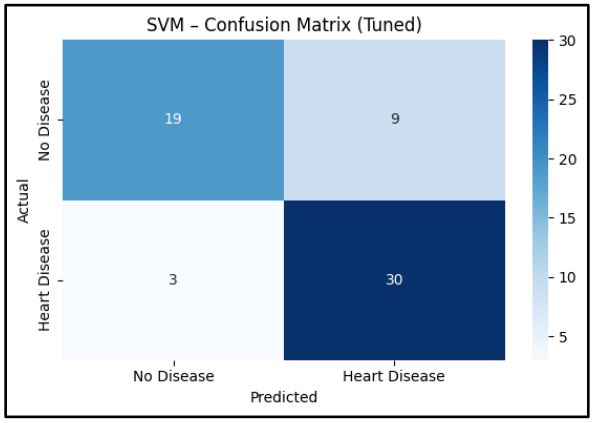


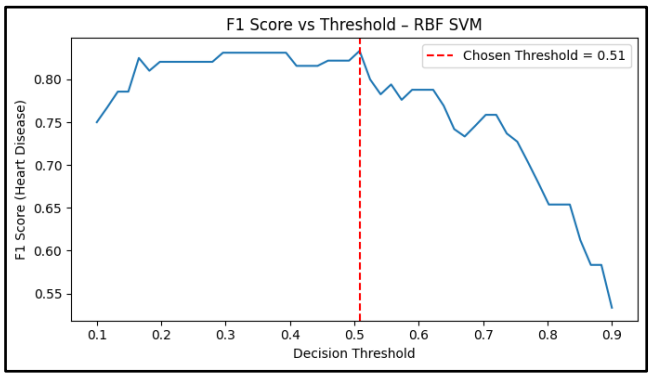
9. Code and Output – With Tuning

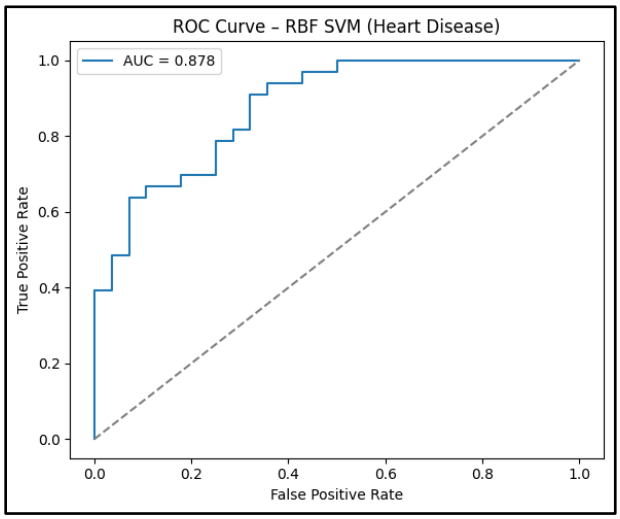
```
# Support Vector Machine – With
Hyperparameter Tuning
(RandomizedSearchCV)
# Dataset: Heart Disease UCI (heart.csv
from Kaggle)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

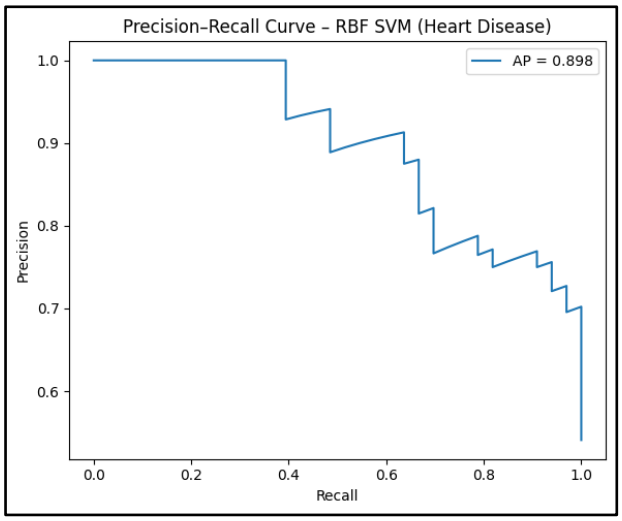
```
from sklearn.model_selection import
train_test_split, RandomizedSearchCV
from sklearn.preprocessing import
StandardScaler
from sklearn.svm import SVC
from sklearn.calibration import
CalibratedClassifierCV
from sklearn.metrics import
(accuracy_score, classification_report,
                        confusion_matrix,
f1_score,
                        roc_curve, auc,
                        precision_recall_curv
e, average_precision_score)
from scipy.stats import loguniform

# Load and clean dataset
df = pd.read_csv("heart.csv")
df = df.replace("?", np.nan)
df = df.apply(pd.to_numeric,
errors="coerce")
df =
df.fillna(df.mean(numeric_only=True))

target_col = "target"
X = df.drop(target_col, axis=1)
y = df[target_col]

# Same split as baseline
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2, stratify=y,
random_state=42
)

scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled  =
scaler.transform(X_test)

# Hyperparameter search — log-uniform
distributions for C and gamma
param_dist = {
    "C":     loguniform(1e-2, 1e2),
    "gamma": loguniform(1e-4, 1e1)
}

base_svm = SVC(kernel='rbf',
class_weight='balanced',
              probability=True,
random_state=42)

random_search =
RandomizedSearchCV(
    estimator=base_svm,
    param_distributions=param_dist,
    n_iter=20,
    cv=3,
    scoring='f1',
    n_jobs=-1,
    verbose=1,
    random_state=42
)

print("Tuning RBF SVM...")
random_search.fit(X_train_scaled,
y_train)

print("\nBest Parameters:",
random_search.best_params_)
print(f"Best CV F1 Score:
{random_search.best_score_:.4f}")

best_svm =
random_search.best_estimator_

# Probability calibration
calibrated_svm =
CalibratedClassifierCV(best_svm,
method='sigmoid', cv=3)
calibrated_svm.fit(X_train_scaled,
y_train)

# Probability estimates for test set
y_probs =
calibrated_svm.predict_proba(X_test_sc
aled)[:, 1]

# Decision threshold optimization
(maximize F1 for heart disease class)
thresholds = np.linspace(0.1, 0.9, 50)
```

```
f1_scores  = [f1_score(y_test, (y_probs
>= t).astype(int)) for t in thresholds]

best_idx       = int(np.argmax(f1_scores))
best_threshold = thresholds[best_idx]
print(f"\nBest Threshold (by F1):
{best_threshold:.3f}")
print(f"Best F1 at this threshold:
{f1_scores[best_idx]:.4f}")

# Final predictions with optimized
threshold
y_pred = (y_probs >=
best_threshold).astype(int)

# Evaluation
print("\n========== SVM (With
Tuning) ==========")
print(f"Accuracy:
{accuracy_score(y_test, y_pred) *
100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test,
y_pred,
      target_names=["No Heart Disease",
"Heart Disease"]))

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# Confusion Matrix plot
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
            xticklabels=["No Disease",
"Heart Disease"],
            yticklabels=["No Disease",
"Heart Disease"])
plt.title('SVM – Confusion Matrix
(Tuned)')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.show()

# ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_probs)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, label=f"AUC =
{roc_auc:.3f}")
plt.plot([0, 1], [0, 1], linestyle='--',
color='gray')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve – RBF SVM (Heart
Disease)")
plt.legend()
plt.tight_layout()
plt.show()

# Precision–Recall Curve
precision, recall, _ =
precision_recall_curve(y_test, y_probs)
avg_precision =
average_precision_score(y_test,
y_probs)
plt.figure(figsize=(6, 5))
plt.plot(recall, precision, label=f"AP =
{avg_precision:.3f}")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision–Recall Curve – RBF
SVM (Heart Disease)")
plt.legend()
plt.tight_layout()
plt.show()

# F1 Score vs Threshold
plt.figure(figsize=(7, 4))
plt.plot(thresholds, f1_scores)
plt.axvline(x=best_threshold, linestyle='-
-', color='red',
            label=f"Chosen Threshold =
{best_threshold:.2f}")
plt.xlabel("Decision Threshold")
plt.ylabel("F1 Score (Heart Disease)")
plt.title("F1 Score vs Threshold – RBF
SVM")
plt.legend()
plt.tight_layout()
plt.show()
```









10. Conclusion

This experiment demonstrates that a Support Vector Machine with an RBF kernel, combined with StandardScaler preprocessing, class balancing, RandomizedSearchCV tuning, probability calibration, and threshold optimisation, serves as an effective classifier for heart disease prediction on the Heart Disease UCI dataset. The baseline model (default C=1.0, gamma='scale') already achieves a competitive 80.33% accuracy, confirming that RBF SVMs are strong out-of-the-box on small, mixed-feature medical datasets. Hyperparameter tuning reveals that a more regularised configuration (C=0.147, gamma=0.00679) better suits this dataset. While overall accuracy remains unchanged, the tuned model reduces false negatives from 5 to 3 — improving heart disease recall from 0.85 to 0.91. This shift is clinically significant: in a screening application, missing a true heart disease case is considerably more dangerous than an unnecessary follow-up test. The near-default optimal threshold of 0.508 confirms that the calibrated model requires minimal threshold adjustment, and that the SVM's inherent class-balanced training already produces well-calibrated decision probabilities. Collectively, the results highlight the value of tuning for precision–recall trade-off control rather than raw accuracy improvement in medical classification tasks.